Таблицы в pandas

Урок

```
pd.crosstab(genre, month)
month          9   10
genre
повесть        1   2
роман          3   0
фантастика     5   3
```



pandas — библиотека для таблиц: столбцы с именами, строки с метками, в одной таблице и числа, и строки, и даты. Стоит она на numpy — каждый столбец внутри это массив, — так что маски, счёт вдоль оси и правило «без цикла» из урока «Массивы в numpy» работают и здесь. Пример на весь урок — школьная библиотека: каталог книг, список читателей и журнал выдач, три файла. Как и в уроке о numpy, код — в цветных ячейках, под чертой — то, что он печатает, а ячейки идут подряд, как в ноутбуке.

1 Таблица и столбец

Таблица в pandas — `DataFrame`. Её можно собрать и руками, из словаря «имя столбца — список значений»:

```
import numpy as np
import pandas as pd
b = pd.DataFrame({
    "title": ["Солярис", "Шинель", "Собачье сердце"],
    "year": [1961, 1842, 1925],
})
print(b)
print(b.shape)
```

```
            title  year
0         Солярис  1961
1          Шинель  1842
2  Собачье сердце  1925
(3, 2)
```

Слева — индекс, метки строк; если их не дать, это номера с нуля. Форма — как у массива: три строки, два столбца. Столбец берут по имени в квадратных скобках:

```
y = b["year"]
print(y)
print(type(y).__name__, y.to_numpy())
```

```
0    1961
1    1842
2    1925
Name: year, dtype: int64
Series [1961 1842 1925]
```

Столбец — это `Series`: массив numpy, к которому приложены индекс и имя. `to_numpy()` достаёт сам массив.
Арифметика — как в numpy, поэлементно, и результат можно записать новым столбцом:

```
b["age"] = 2026 - b["year"]
print(b)
print(b["age"].mean(), b["age"].max())
```

```
             title  year  age
0         Солярис  1961   65
1          Шинель  1842  184
2  Собачье сердце  1925  101
116.66666666666667 184
```

У каждого столбца свой тип — поэтому строки и числа уживаются в одной таблице:

```
print(b.dtypes)
```

```
title      str
year     int64
age      int64
dtype: object
```

`str` — строки, `int64` — целые, как в numpy. Строка `object` в конце — тип самого списка типов, его можно не замечать.

Пример (самая старая). Какая из книг старше всех. `idxmax` возвращает не наибольшее значение, а метку его строки — как `argmax` в numpy, только метку, а не номер:

```
i = b["age"].idxmax()
print(i, b.loc[i, "title"])
```

```
1 Шинель
```

`b.loc[метка, столбец]` — одна клетка таблицы по метке строки и имени столбца. Подробно о `loc` — в разделе 4.

2 Чтение файла

Журнал выдач выгружен из Excel. Прежде чем читать файл, стоит посмотреть, что в нём на самом деле, — первые строки как текст:

```
with open("vydachi.csv", encoding="utf-8") as f:
    for line in f.readlines()[:4]:
        print(line, end="")
```

```
reader;book;taken;returned;fine
r1;k01;01.09.2026;12.09.2026;0
r2;k03;01.09.2026;25.09.2026;12,5
r3;k02;03.09.2026;10.09.2026;0
```

Разделитель — точка с запятой, дробная часть — после запятой, даты — день.месяц.год: так пишет русский Excel. А `read_csv` по умолчанию ждёт разделитель-запятую. Здесь

он находит запятую в штрафе 12,5 и не понимает, откуда в строке второе поле:

```
loans = pd.read_csv("vydachi.csv")
```

```
ParserError: Error tokenizing data. C error: Expected
1 fields in line 3, saw 2
```

Разделитель и знак дробной части задают параметрами:

```
loans = pd.read_csv("vydachi.csv",
                    sep=";", decimal=",")
print(loans.head())
print(loans.dtypes)
```

```
  reader book       taken    returned  fine
0     r1  k01  01.09.2026  12.09.2026   0.0
1     r2  k03  01.09.2026  25.09.2026  12.5
2     r3  k02  03.09.2026  10.09.2026   0.0
3     r1  k05  12.09.2026  20.09.2026   0.0
4     r4  k01  14.09.2026  30.09.2026   5.0
reader          str
book            str
taken           str
returned        str
fine        float64
dtype: object
```

`head()` — первые пять строк, `tail()` — последние. Штраф стал дробным числом, пустые места — `NaN`, пропусками, как

в numpy. Даты пока строки: о них раздел 5. Каталог — обычный файл с запятыми:

```
books = pd.read_csv("knigi.csv")
print(books)
```

```
  book                 title         genre  year
0  k01    Пикник на обочине    фантастика  1972
1  k02  Мастер и Маргарита         роман  1967
2  k03              Солярис    фантастика  1961
3  k04       Собачье сердце       повесть  1925
4  k05   Трудно быть богом   Фантастика   1964
5  k06   Капитанская дочка         роман  1836
6  k07                Шинель       повесть  1842
```

Ловушка. Список читателей выгружен из старой программы, и не в UTF-8:

```
readers = pd.read_csv("chitateli.csv")
```

```
UnicodeDecodeError: 'utf-8' codec can't decode
byte 0xc0 in position 0: invalid start byte
```

Кириллица в Windows бывает в кодировке cp1251, и UTF-8 её не читает. Кодировку называют прямо:

```
readers = pd.read_csv("chitateli.csv",
                      encoding="cp1251")
print(readers.head(3))
```

```
  reader  name grade
0     R1   Аня   10А
1     R2  Боря   10Б
2     R3  Вера   10А
```

Хуже, когда ошибки нет, а вместо букв — мусор вроде «РђPSCЦ»: это UTF-8, прочитанный как cp1251. Поэтому после чтения — всегда `head()` глазами.

3 Первый взгляд и чистка

Сначала — сколько строк, где пропуски и что в числовых столбцах:

```
print(loans.shape)
print(loans.isna().sum())
print(loans["fine"].describe().round(2))
```

```
(16, 5)
reader      0
book        0
taken       0
returned    6
fine        6
dtype: int64
count    10.00
mean      2.50
std       4.41
min       0.00
25%       0.00
50%       0.00
75%       3.75
max      12.50
Name: fine, dtype: float64
```

`isna()` — маска пропусков, как `np.isnan`, а сумма по столбцам — сколько их. В шести строках нет ни даты возврата, ни штрафа: эти книги ещё на руках. Одна из шести, как выяснится ниже, — повтор. `describe` даёт сводку по столбцу: сколько значений, среднее, разброс, наименьшее,

квартили, наибольшее. У столбца строк лучше смотреть, какие значения встречаются:

```
print(books["genre"].value_counts())
```

```
genre
фантастика     2
роман          2
повесть        2
Фантастика     1
Name: count, dtype: int64
```

«Фантастика » с большой буквы и пробелом на конце — для pandas другое значение, чем «фантастика». Строковые методы на весь столбец сразу — через `.str`:

```
g = books["genre"].str.strip().str.lower()
books["genre"] = g
print(books["genre"].value_counts())
```

```
genre
фантастика    3
роман         2
повесть       2
Name: count, dtype: int64
```

`strip` снимает пробелы по краям, `lower` переводит в строчные. Ещё `.str.replace`, `.str.contains`, `.str.startswith`, `.str.len` — всё, что умеют строки питона, но на весь столбец. Если подходящего метода нет, пригодится `apply` — раздел 9.

Теперь повторы. `duplicated` отмечает строку, если точно такая же уже была выше; `keep=False` отмечает все копии:

```
print(loans.duplicated().sum())
print(loans[loans.duplicated(keep=False)])
loans = loans.drop_duplicates()
print(len(loans))
```

```
1
   reader book       taken returned  fine
11     r3  k07  10.10.2026      NaN   NaN
14     r3  k07  10.10.2026      NaN   NaN
15
```

Ловушка. Счёт в pandas пропуски пропускает сам — в отличие от numpy, где `np.mean` с `nan` даёт `nan`:

```
print(loans["fine"].sum(), loans["fine"].mean())
print(loans["fine"].fillna(0).mean())
```

```
25.0 2.5
1.6666666666666667
```

Среднее вышло по десяти известным штрафам, а не по всем выдачам. Какое нужно — решает задача; `fillna` заполняет пропуски заданным значением, и тогда считаются все строки.

Бывает и хуже: в числовом столбце попадается текст. `pd.to_numeric` с `errors="coerce"` превращает всё, что не число, в пропуск:

```
raw = pd.Series(["12", "7,5", "нет", ""])
fixed = raw.str.replace(",", ".")
print(pd.to_numeric(fixed, errors="coerce"))
```

```
0    12.0
1     7.5
2     NaN
3     NaN
dtype: float64
```

Без `errors="coerce"` он упадёт на первом же «нет» — и это тоже полезно: сразу видно, что в столбце не только числа.

4 Выбор строк и столбцов

Маски — как в numpy: сравнение столбца даёт `Series` из `True` и `False`, и таблица в квадратных скобках оставляет строки с `True`. Список имён в квадратных скобках оставляет столбцы:

```
print(loans[loans["fine"] > 0])
print(loans[["reader", "book"]].head(3))
```

```
  reader book       taken    returned  fine
1     r2  k03  01.09.2026  25.09.2026  12.5
4     r4  k01  14.09.2026  30.09.2026   5.0
7     r5  k06  27.09.2026  11.10.2026   7.5
  reader book
0     r1  k01
1     r2  k03
2     r3  k02
```

Условия — те же `&`, `|`, `~` в скобках. Для «равно одному из» — `isin`, для «между» — `between`, концы входят:

```
r1 = loans[(loans["reader"] == "r1")
           & loans["returned"].isna()]
print(r1)
print(loans["book"].isin(["k01", "k03"]).sum())
print(books[books["year"].between(1900, 1970)])
```

```
   reader book       taken returned  fine
12     r1  k01  06.10.2026      NaN   NaN
6
  book               title       genre  year
1  k02  Мастер и Маргарита       роман  1967
2  k03             Солярис  фантастика  1961
3  k04      Собачье сердце     повесть  1925
4  k05   Трудно быть богом  фантастика  1964
```

Строки и столбцы сразу выбирает `loc`: `loc[строки, столбцы]`, где строки — метки, срез меток или маска. `iloc` — то же по номерам, как в numpy:

```
fined = loans["fine"] > 0
print(loans.loc[fined, ["reader", "fine"]])
print(loans.iloc[:2, :3])
```

```
  reader  fine
1     r2  12.5
4     r4   5.0
7     r5   7.5
  reader book       taken
0     r1  k01  01.09.2026
1     r2  k03  01.09.2026
```

Ловушка. Метки и номера строк — не одно и то же. После `drop_duplicates` в индексе дыра: строки с меткой 14 больше нет.

```
print(loans.index[-3:].tolist())
print(loans.iloc[-1]["book"])
print(loans.loc[15, "book"])
```

```
[12, 13, 15]
k08
k08
```

`iloc[-1]` — последняя строка по счёту, `loc[15]` — строка с меткой 15; здесь это одно и то же, но `loc[14]` упадёт, а `iloc[14]` вернёт строку с меткой 15. Пронумеровать заново — `reset_index(drop=True)`.

Записывают тоже через `loc`: строки маской, столбец по имени. Запись цепочкой из двух квадратных скобок не работает:

```
t = loans.copy()
t[t["fine"] > 0]["fine"] = 0
print(t["fine"].max())
```

```
12.5
ChainedAssignmentError: A value is being set on a
copy of a DataFrame or Series through chained
assignment.
```

Штраф 12,5 на месте. А через `loc` — записалось:

```
t.loc[t["fine"] > 0, "fine"] = 0
print(t["fine"].max())
```

```
0.0
```

Как устроено. `t[маска]` — новая таблица, и `["fine"] = 0` пишет в неё, а не в `t`; pandas 3 об этом предупреждает. Общее правило pandas 3: всё, что получено

из таблицы, — столбец, срез, выбор маской, — ведёт себя как отдельная копия. Это противоположно срезам numpy и избавляет от их ловушки, зато менять таблицу можно только прямо: `t["col"] = ...` или `t.loc[...] = ...`.

5 Даты

Даты в файле — строки. `pd.to_datetime` превращает их в настоящие даты, и формат лучше назвать: `%d` — день, `%m` — месяц, `%Y` — год четырьмя цифрами.

```
fmt = "%d.%m.%Y"
for c in ["taken", "returned"]:
    loans[c] = pd.to_datetime(loans[c], format=fmt)
print(loans.dtypes)
print(loans.head(3))
```

```
reader                 str
book                   str
taken       datetime64[us]
returned    datetime64[us]
fine               float64
dtype: object
  reader book      taken   returned  fine
0     r1  k01 2026-09-01 2026-09-12   0.0
1     r2  k03 2026-09-01 2026-09-25  12.5
2     r3  k02 2026-09-03 2026-09-10   0.0
```

Ловушка. Без формата pandas угадывает его по первой дате, и угадывает по-американски — месяц первым. Если

все дни не больше 12, ошибки не будет, и даты молча станут другими:

```
s = pd.Series(["03.09.2026", "05.10.2026"])
print(pd.to_datetime(s))
```

```
0   2026-03-09
1   2026-05-10
dtype: datetime64[us]
```

Третье сентября стало девятым марта. Формат — всегда.

Пропуски у дат свои — `NaT`, «не время». Разность двух дат — промежуток, а `.dt` открывает части дат и промежутков, как `.str` у строк:

```
dur = loans["returned"] - loans["taken"]
loans["days"] = dur.dt.days
print(loans[["reader", "taken", "days"]].head())
print(loans["taken"].dt.month.value_counts())
```

```
  reader      taken  days
0     r1 2026-09-01  11.0
1     r2 2026-09-01  24.0
2     r3 2026-09-03   7.0
3     r1 2026-09-12   8.0
4     r4 2026-09-14  16.0
taken
9     9
10    6
Name: count, dtype: int64
```

`dt.days` — промежуток в днях; там, где книгу не вернули, `NaN`. Ещё `dt.year`, `dt.day`, `dt.dayofweek` — день недели,

0 — понедельник. Даты сравнивают с датами или со строкой вида `2026-10-01` — год, месяц, день:

```
today = pd.Timestamp("2026-10-20")
out = loans[loans["returned"].isna()]
print((today - out["taken"]).dt.days.tolist())
print((loans["taken"] >= "2026-10-01").sum())
```

```
[24, 19, 10, 14, 8]
6
```

Первая строка — сколько дней на руках книги, которые ещё не вернули, на 20 октября; вторая — сколько выдач было с первого октября.

6 Сортировка

`sort_values` — по столбцу, по возрастанию; `ascending=False` — по убыванию. Пропуски уходят в конец:

```
cols = ["reader", "book", "taken", "days"]
s = loans.sort_values("days", ascending=False)
print(s[cols])
```

```
   reader book      taken  days
1      r2  k03 2026-09-01  24.0
4      r4  k01 2026-09-14  16.0
7      r5  k06 2026-09-27  14.0
0      r1  k01 2026-09-01  11.0
8      r1  k02 2026-09-28   8.0
3      r1  k05 2026-09-12   8.0
10     r2  k01 2026-10-02   7.0
2      r3  k02 2026-09-03   7.0
13     r5  k05 2026-10-08   7.0
5      r2  k07 2026-09-25   3.0
6      r3  k03 2026-09-26   NaN
9      r4  k04 2026-10-01   NaN
11     r3  k07 2026-10-10   NaN
```

```
12      r1  k01 2026-10-06   NaN
15      r2  k08 2026-10-12   NaN
```

Ловушка. При равных значениях порядок по одному столбцу не обещан — как у `argsort` без `kind="stable"`. Нужен порядок равных — второй столбец; направление у каждого своё:

```
s = loans.sort_values(["days", "taken"],
                      ascending=[False, True])
print(s[cols].head(5))
```

```
  reader book      taken  days
1     r2  k03 2026-09-01  24.0
4     r4  k01 2026-09-14  16.0
7     r5  k06 2026-09-27  14.0
0     r1  k01 2026-09-01  11.0
3     r1  k05 2026-09-12   8.0
```

Несколько наибольших — `nlargest`, наименьших — `nsmallest`; равные они берут в порядке таблицы:

```
print(books.nsmallest(2, "year"))
```

```
  book              title    genre  year
5  k06  Капитанская дочка    роман  1836
6  k07             Шинель  повесть  1842
```

7 Группы

`groupby` делит таблицу на группы по значениям столбца и считает по каждой. `size()` — сколько строк в группе, а столбец с функцией — что угодно:

```
g = loans.groupby("reader")
print(g.size())
```

```
reader
r1    4
r2    4
r3    3
r4    2
r5    2
dtype: int64
```

```
print(g["fine"].sum())
```

```
reader
r1     0.0
r2    12.5
r3     0.0
r4     5.0
r5     7.5
Name: fine, dtype: float64
```

Результат — `Series`, где индекс — значения групп. Несколько счётов сразу — `agg`, каждый под своим именем: имя столбца результата, затем пара «столбец таблицы, функция»:

```
print(loans.groupby("reader").agg(
    n=("book", "size"),
    books=("book", "nunique"),
    fine=("fine", "sum"),
    last=("taken", "max")))
```

```
        n  books  fine       last
reader
r1      4      3   0.0 2026-10-06
r2      4      4  12.5 2026-10-12
r3      3      3   0.0 2026-10-10
r4      2      2   5.0 2026-10-01
r5      2      2   7.5 2026-10-08
```

`nunique` — сколько различных значений: Аня, `r1`, брала книги четыре раза, но разных у неё три. Для подсчёта значений одного столбца есть и короткий путь, уже знакомый:

```
print(loans["book"].value_counts().head(3))
```

```
book
k01    4
k03    2
k02    2
Name: count, dtype: int64
```

Группировать можно и по двум столбцам — тогда группа — пара значений, и индекс результата из двух уровней:

```
m = loans["taken"].dt.month
print(loans.groupby(["reader", m]).size().head(4))
```

```
reader  taken
r1      9        3
        10       1
r2      9        2
        10       2
dtype: int64
```

Значение группы бывает нужно в каждой строке, а не по строке на группу. Это `transform`: результат той же длины, что таблица. Насколько каждая выдача дольше обычной для этого читателя:

```
by_r = loans.groupby("reader")
avg = by_r["days"].transform("mean")
diff = (loans["days"] - avg).round(1)
print(diff.head(6).tolist())
```

```
[2.0, 12.7, 0.0, -1.0, 0.0, -8.3]
```

Как устроено. `groupby` — это `unique` и `bincount` из урока о numpy: значения столбца переводятся в номера групп, и счёт идёт за один проход по данным. Поэтому группировка быстрая, а цикл по группам с выбором маской — медленный: он проходит таблицу столько раз, сколько групп.

8 Соединение таблиц

Названия книг — в каталоге, а в журнале только коды. `merge` ставит рядом строки двух таблиц с одинаковым ключом:

```
full = loans.merge(books, on="book", how="left")
print(full[["reader", "title", "genre"]].head())
```

```
  reader                title       genre
0     r1   Пикник на обочине  фантастика
1     r2             Солярис  фантастика
2     r3  Мастер и Маргарита       роман
3     r1    Трудно быть богом  фантастика
4     r4   Пикник на обочине  фантастика
```

`how="left"` оставляет все строки левой таблицы, журнала, даже если в каталоге пары не нашлось. Таких строк быть не должно — проверим: `indicator=True` добавляет столбец, откуда строка.

```
chk = loans.merge(books, on="book", how="left",
                  indicator=True)
print(chk["_merge"].value_counts())
print(chk.loc[chk["_merge"] == "left_only", "book"])
```

```
_merge
both          14
left_only      1
right_only     0
Name: count, dtype: int64
14    k08
Name: book, dtype: str
```

Книги `k08` в каталоге нет. Это ошибка в данных, и её видно только так: при `how="inner"`, который стоит по умолчанию, строка тихо пропала бы.

Ловушка. Ключи, записанные по-разному, не совпадают. В журнале читатели `r1`, `r2`, а в списке — `R1`, `R2`:

```
print(len(loans.merge(readers, on="reader")))
readers["reader"] = readers["reader"].str.lower()
print(len(loans.merge(readers, on="reader")))
```

```
0
15
```

Ноль строк и никакой ошибки. Ключи приводят к одному виду до соединения: регистр, пробелы, ведущие нули, тип — строка `"7"` не равна числу 7.

Соединять можно и с другой стороны. Сколько книг брал каждый читатель, включая тех, кто не брал ни одной:

```
r = readers.merge(loans, on="reader", how="left")
print(r.groupby("name")["book"].count())
```

```
name
Аня     4
Боря    4
Вера    3
Гоша    2
Даша    2
Егор    0
Name: book, dtype: int64
```

У Егора выдач нет, и `left` оставил его строку с пропусками. `count` считает только непустые значения — вышел ноль. А `size` посчитал бы строки и дал бы единицу.

Пример (любимый жанр). Какие жанры брали десятиклассники из 10А. Три таблицы вместе, маска по классу, подсчёт значений:

```
x = full.merge(readers, on="reader")
a10 = x.loc[x["grade"] == "10А", "genre"]
print(a10.value_counts())
```

```
genre
фантастика    4
роман         2
повесть       1
Name: count, dtype: int64
```

Ещё одна опасность — повторы ключа справа: если бы книга `k01` дважды стояла в каталоге, каждая её выдача удвоилась бы. Параметр `validate="many_to_one"` проверяет, что справа ключ не повторяется, и падает, если повторяется.

9 Своя функция: apply

Бывает, что готового метода нет. Тогда функцию пишут сами, а pandas вызывает её: `s.apply(f)` подставляет в `f` каждый элемент столбца и собирает ответы в новый столбец. Подпишем авторов каталога фамилией и инициалом:

```
def short(name):
    first, last = name.split()
    return last + " " + first[0] + "."

av = pd.Series(["Аркадий Стругацкий",
                "Михаил Булгаков", "Станислав Лем"])
print(av.apply(short).tolist())
```

```
['Стругацкий А.', 'Булгаков М.', 'Лем С.']
```

`apply` — обычный цикл по столбцу: питон вызывает `f` по разу на элемент, и всё, что умеет питон, умеет `f`. Но цикл

медленный, а правил у него несколько, — о них ниже. Поэтому сначала проверяют, нет ли замены без цикла. Их четыре:

- арифметика и сравнения — прямо на столбце: `fine * 2` вместо `fine.apply(lambda v: v * 2)`;
- «если, то, иначе» по числам — `np.where`, по интервалам — `pd.cut`;
- подстановка по таблице значений — `.map(словарь)`;
- действия со строками — методы `.str`.

Первые две известны по numpy. Условие через `apply` и через `np.where` дают один и тот же столбец:

```
fine = loans["fine"]
t = pd.DataFrame({"fine": fine})
t["apply"] = fine.apply(lambda v: 10 if v > 5 else 0)
t["where"] = np.where(fine > 5, 10, 0)
print(t.head(4))
```

```
   fine  apply  where
0   0.0      0      0
1  12.5     10     10
2   0.0      0      0
3   0.0      0      0
```

Подстановка по таблице значений — `.map`. Жанра «повесть» в словаре нет, и на его месте остаётся пропуск; а `apply(lambda v: code[v])` упал бы с `KeyError`:

```
code = {"фантастика": "Ф", "роман": "Р"}
print(books["genre"].map(code).tolist())
```

```
['Ф', 'Р', 'Ф', nan, 'Ф', 'Р', nan]
```

Когда же `apply` нужен? Когда ни одна замена не ложится: функция с ветвлением и разбором строки по частям, как `short`; решение, которое зависит от нескольких столбцов

одной строки; счёт по всей подтаблице группы. И когда строк не миллионы: на тысяче строк разницы не заметить.

У таблицы `apply` вызывает функцию не на элементе, а на целой строке или целом столбце. Посчитаем статус каждой выдачи по нескольким столбцам сразу; `axis=1` значит «по строкам»:

```
def status(row):
    if pd.isna(row["returned"]):
        return "на руках"
    if row["fine"] > 0:
        return "вернул со штрафом"
    return "вернул"

st = loans.apply(status, axis=1)
print(st.value_counts())
```

```
вернул               7
на руках             5
вернул со штрафом    3
Name: count, dtype: int64
```

Ловушка. Без `axis=1` функция получит не строку, а столбец: по умолчанию `df.apply` идёт по столбцам (`axis=0`). Код, написанный под строку, падает: в столбце метки — номера строк, а имени `fine` среди них нет:

```
loans.apply(lambda r: r["fine"] * 2)
```

```
KeyError: 'fine'
```

Для сводки по каждому столбцу умолчание как раз то, что нужно.

Ловушка. При `axis=1` pandas собирает каждую строку в одну `Series`, а у неё тип один на всех. Если в таблице

целые и дробные числа, строка выходит дробной, и целый номер приходит как `101.0`:

```
cards = pd.DataFrame({"card": [101, 102, 103],
                      "fine": [0.0, 12.5, 5.0]})

def line(r):
    return f"{r['card']}: {r['fine']}"

print(cards.apply(line, axis=1).tolist())
```

```
['101.0: 0.0', '102.0: 12.5', '103.0: 5.0']
```

Добавим столбец со строками — строка станет `object`, и номер придёт целым:

```
cards["who"] = ["Аня", "Боря", "Вера"]
print(cards.apply(line, axis=1).tolist())
```

```
['101: 0.0', '102: 12.5', '103: 5.0']
```

Одна и та же функция, а ответ зависит от соседних столбцов. Надёжнее не полагаться на тип: `int(r["card"])` в самой функции или вовсе работа со столбцами, без `axis=1`.

Ловушка. Форма результата при `axis=1` зависит от того, что вернула функция. Число даёт `Series`; список — тоже `Series`, но из списков; а `Series` внутри даёт уже `DataFrame`:

```
c = cards[["card", "fine"]]
one = c.apply(lambda r: r["fine"] * 2, axis=1)
lst = c.apply(lambda r: [r["fine"], 0], axis=1)
tab = c.apply(lambda r: r[["fine"]] * 2, axis=1)
print(one.shape, lst.shape, tab.shape)
```

```
(3,) (3,) (3, 1)
```

Разложить список по столбцам просит `result_type="expand"`:

```
e = c.apply(lambda r: [r["fine"], 0], axis=1,
            result_type="expand")
print(e.shape)
```

```
(3, 2)
```

Что вернула функция, то pandas и склеил, поэтому форму результата стоит проверить, а не угадывать.

Ловушка. `s.apply(f)` отдаёт в `f` элементы питоновскими объектами, а не тем, что лежит в массиве numpy: число приходит как `float`, а не `float64`.

```
z = pd.Series([1.0, 0.0, 10.0])
print(type(z.iloc[0]).__name__)
print(z.apply(lambda v: type(v).__name__).iloc[0])
```

```
float64
float
```

Поэтому функции из `math` работают, но не прощают того, что прощает numpy:

```
import math
z.apply(math.log)
```

```
ValueError: math domain error
```

А функции numpy, ufunc вроде `np.log` и `np.sqrt`, `apply` поэлементно не вызывает: ему передают весь столбец сразу, как в `np.log(z)`. Нуль даёт `-inf`, а не ошибку:

```
print(z.apply(np.log).tolist())
```

```
[0.0, -inf, 2.302585092994046]
```

Ловушка. Пропуск приходит в функцию как `nan` — обычное число, а не строка. Функция для строк на нём падает, а метод `.str` пропуск обходит сам:

```
nm = pd.Series(["Аркадий Стругацкий", None])
nm.apply(short)
```

```
AttributeError: 'float' object has no attribute
'split'
```

Свою функцию от пропусков бережёт `map` с `na_action="ignore"`:

```
print(nm.map(short, na_action="ignore").tolist())
print(nm.str.upper().tolist())
```

```
['Стругацкий А.', nan]
['АРКАДИЙ СТРУГАЦКИЙ', nan]
```

Ловушка. `groupby().apply(f)` отдаёт в `f` подтаблицу группы. В pandas 3 в ней уже нет столбца, по которому группировали; в pandas 2 он был (с версии 2.2 pandas предупреждал, что так перестанет быть). Код под одну версию на другой ведёт себя иначе:

```
def cols(grp):
    return grp.columns.tolist()

print(loans.groupby("reader").apply(cols).iloc[0])
```

```
['book', 'taken', 'returned', 'fine', 'days']
```

```
loans.groupby("reader").apply(
    lambda grp: grp["reader"].iloc[0])
```

```
KeyError: 'reader'
```

Ключ группы лежит в `grp.name` — и там, и там. А чтобы поведение не зависело от версии вовсе, пишут `include_groups=False` (с pandas 2.2): столбца группировки тогда нет и в pandas 2. Если из группы нужно число, а не таблица, проще и быстрее `agg` и `transform`.

Как устроено (скорость). Цикл `apply` идёт на питоне: на каждый элемент — вызов функции. Векторная операция — один вызов на весь столбец, а цикл внутри numpy, на C. Замер на миллионе чисел:

```
import time

def clock(f):
    t = time.perf_counter()
    f()
    return time.perf_counter() - t

x = pd.Series(np.arange(10.0**6))
a = clock(lambda: x.apply(lambda v: v * 2 + 1))
b = clock(lambda: x * 2 + 1)
print(f"apply: {a:.3f} с, столбец: {b:.4f} с")
```

```
apply: 0.244 с, столбец: 0.0071 с
```

То же по строкам, `axis=1`, на ста тысячах строк:

```
d = pd.DataFrame({"a": x[:10**5], "b": x[:10**5]})
a = clock(lambda: d.apply(lambda r: r["a"] + r["b"],
                          axis=1))
b = clock(lambda: d["a"] + d["b"])
print(f"по строкам: {a:.2f} с, столбцы: {b:.5f} с")
```

```
по строкам: 0.60 с, столбцы: 0.00059 с
```

И ufunc из ловушки выше — один раз как есть, другой раз в обёртке:

```
y = pd.Series(np.arange(1.0, 10.0**6))
a = clock(lambda: y.apply(np.log))
b = clock(lambda: y.apply(lambda v: np.log(v)))
print(f"np.log: {a:.3f} с, в обёртке: {b:.3f} с")
```

```
np.log: 0.006 с, в обёртке: 0.381 с
```

Для чисел `apply` проигрывает в десятки раз, а по строкам — в сотни и тысячи: на каждую строку он собирает `Series`. И `apply(np.log)` быстр лишь потому, что идёт целым столбцом: стоит обернуть в `lambda`, и это снова миллион вызовов.

Со строками картина другая, и «`.str` быстрее» верно не всегда. Без pyarrow pandas 3 хранит строки как питоновские объекты, и метод `.str` внутри — тоже цикл по ним. Фамилия с инициалом через `.str` выходит цепочкой: разбить, взять первую букву, склеить. Сравним с `short` на шестистах тысячах строк:

```
def chain(a):
    p = a.str.split(expand=True)
    return p[1] + " " + p[0].str[0] + "."

print(av.dtype.storage)
print(chain(av).equals(av.apply(short)))
big = pd.Series(list(av) * 200000)
a = clock(lambda: big.apply(short))
b = clock(lambda: chain(big))
print(f"apply: {a:.2f} с, цепочка: {b:.2f} с")
```

```
python
True
apply: 0.37 с, цепочка: 1.33 с
```

Цепочка проходит столбец заново на каждом шаге и каждый раз строит новый столбец, поэтому одна функция в `apply` её обгоняет. Но виновата цепочка, а не `.str`: простые операции — `upper`, `replace`, `contains`, `startswith`, `len` — не медленнее `apply`, а `len` даже в разы быстрее. С pyarrow (`pip install pyarrow`; строки тогда хранит колонка Arrow, и `storage` будет `pyarrow`) эти простые методы при замере на 500 000 строк обгоняли `apply` в 5–75 раз. Но `split` и `extract` легче не стали: цепочка с `split` и там осталась медленнее одного `apply` (0,9 с против 0,4 с).

> **Правило.** Для чисел — операция на столбце, `np.where`, `.map`; `apply` там почти всегда лишний. Для строк не гадать, а замерять на своих данных: одна операция `.str` обычно не хуже `apply`, цепочка из нескольких может проиграть. Взялись за `apply` — проверьте три вещи: `axis`, тип того, что пришло в функцию, и форму того, что она вернула.

10 Сдвиги и окна

`shift` сдвигает столбец на строку вниз: в каждой строке — значение из предыдущей. Внутри групп — через `groupby`. Сколько дней прошло между выдачами одного читателя:

```
loans = loans.sort_values(["reader", "taken"])
prev = loans.groupby("reader")["taken"].shift()
loans["gap"] = (loans["taken"] - prev).dt.days
print(loans[["reader", "taken", "gap"]].head(6))
```

```
   reader      taken   gap
0      r1 2026-09-01   NaN
3      r1 2026-09-12  11.0
8      r1 2026-09-28  16.0
12     r1 2026-10-06   8.0
1      r2 2026-09-01   NaN
5      r2 2026-09-25  24.0
```

У первой выдачи каждого читателя предыдущей нет — `NaN`. Сдвиг смотрит на соседнюю строку, поэтому сначала — сортировка, иначе «предыдущая» значит «выше в файле».

Окна — для ряда по времени. Сколько выдач в каждый день: подсчёт дат, и пустые дни дописать нулями, иначе их в ряду просто нет:

```
per_day = loans["taken"].value_counts().sort_index()
days = pd.date_range("2026-09-01", "2026-10-15")
per_day = per_day.reindex(days, fill_value=0)
print(per_day.head(4))
```

```
2026-09-01    2
2026-09-02    0
2026-09-03    1
2026-09-04    0
Freq: D, Name: count, dtype: int64
```

`date_range` — все дни подряд, `reindex` — переставить ряд по новому индексу, недостающие — нулём. Теперь `rolling` — окно из семи строк, то есть дней:

```
w = per_day.rolling(7).sum()
print(w.head(8).tolist())
w1 = per_day.rolling(7, min_periods=1).sum()
print(w1.head(8).tolist())
```

```
[nan, nan, nan, nan, nan, nan, 3.0, 1.0]
[2.0, 2.0, 3.0, 3.0, 3.0, 3.0, 3.0, 1.0]
```

Ловушка. У первых шести дней полного окна нет, и `rolling(7)` даёт там `NaN`. `min_periods=1` считает и по неполному окну — сколько дней есть. Что из этого правильно, решает задача: «за последние семь дней» в начале ряда — это или пропуск, или «за сколько есть».

Ещё по времени: `resample` собирает ряд по периодам — `"W"` — неделя, по воскресенье, `"MS"` — месяц; `cumsum` — накопленная сумма, как в numpy; `groupby(...).cumcount()` — номер строки внутри группы, с нуля.

```
print(per_day.resample("W").sum().head(3))
print(per_day.cumsum().iloc[-1])
n = loans.groupby("reader").cumcount()
print(n.tolist()[:6])
```

```
2026-09-06    3
2026-09-13    1
2026-09-20    1
Freq: W-SUN, Name: count, dtype: int64
15
[0, 1, 2, 3, 0, 1]
```

11 Сводные таблицы

Сводная таблица — группы по двум столбцам, разложенные строками и столбцами. Самая простая — `crosstab`, подсчёт пар значений:

```
full["month"] = full["taken"].dt.month
print(pd.crosstab(full["genre"], full["month"]))
```

```
month       9   10
genre
повесть      1   2
роман        3   0
фантастика   5   3
```

`pivot_table` считает любую функцию: здесь сумма штрафов по жанрам и месяцам. Пар, которых не было, `fill_value` заполняет нулём:

```
pt = full.pivot_table(index="genre",
                      columns="month",
                      values="fine", aggfunc="sum",
                      fill_value=0)
print(pt)
```

```
month         9    10
genre
повесть      0.0  0.0
роман        7.5  0.0
фантастика  17.5  0.0
```

Жанра у книги `k08` нет — её строки в сводную не попали: группы с пропуском в ключе `groupby` и сводные таблицы отбрасывают. Если они нужны — `dropna=False`.

12 Ответ файлом

Ответ сдают файлом — `to_csv`. Индекс в файл писать не нужно: `index=False`.

```
ans = (full.groupby("genre").size()
           .reset_index(name="n"))
ans.to_csv("otvet.csv", index=False)
with open("otvet.csv", encoding="utf-8") as f:
    print(f.read())
```

```
genre,n
повесть,3
роман,3
фантастика,8
```

`reset_index` превращает индекс — здесь жанры — обратно в столбец, а `name="n"` называет столбец со счётом.

Ловушка. Без `index=False` первым столбцом в файл уйдёт индекс, и у этого столбца нет имени. Прочитанный обратно, файл выглядит так:

```
ans.to_csv("otvet2.csv")
print(pd.read_csv("otvet2.csv").columns.tolist())
```

```
['Unnamed: 0', 'genre', 'n']
```

Проверка ждёт два столбца, а получает три.

Дробные лучше округлить до записи: `round(2)` или параметр `float_format="%.2f"`. Разделитель и порядок столбцов — как просит условие: `sep`, `columns=[...]`.

13 Итог: цепочка

Методы pandas возвращают новую таблицу, поэтому их пишут цепочкой, шаг за шагом, в скобках — и тогда каждую

строку можно читать отдельно. Штрафы по жанрам, от большего:

```
res = (loans
       .merge(books, on="book")
       .query("fine > 0")
       .groupby("genre")["fine"].sum()
       .sort_values(ascending=False))
print(res)
```

```
genre
фантастика    17.5
роман          7.5
Name: fine, dtype: float64
```

`query` — та же маска, записанная строкой: `"fine > 0"` вместо `loans["fine"] > 0`.

> **Правило.** Прочитать — и сразу посмотреть: `head`, `dtypes`, `isna().sum()`, `value_counts` у строк. Половина ошибок в задачах на таблицы — в файле, а не в счёте: разделитель, кодировка, формат дат, разнобой в записи, повторы, ключи, которые не совпали.

Дальше эти таблицы пойдут в модели: sklearn принимает таблицу pandas или массив numpy, где строка — объект, столбец — признак. Как готовить признаки и проверять модель, будет в уроках следующих тем.